

# Warm-up: an email as a type, then five small types

## Programming Paradigms Session 3 — the warm-up, before the quest

Part 1, on paper, in pairs: about 15 minutes. Part 2, at the keyboard with `warmup.ml`: about 20 minutes; the rest at home.

### SOLUTIONS

The lecture said that an algebraic data type is a closed list of shapes, written where the compiler can read it, and that the compiler pays us back by checking every `match`. Before the music language, we try that on something everyone owns: a notification that goes out by email, by SMS or by push. Nothing here is themed, and in Part 1 there is nothing to run. The notation is OCaml's, and these are all we need:

---

|                                              |                                                                                          |
|----------------------------------------------|------------------------------------------------------------------------------------------|
| <code>type t = A   B   C</code>              | a <i>sum</i> : a value of <code>t</code> is A, or B, or C                                |
| <code>type t = A of int   B of string</code> | a sum whose shapes carry something                                                       |
| <code>type t = { f : int; g : bool }</code>  | a <i>product</i> : one value holds an <code>f</code> and a <code>g</code>                |
| <code>int * bool</code>                      | a product without names, a pair                                                          |
| <code>t list, t option</code>                | zero or more <code>t</code> ; a <code>t</code> or nothing ( <code>Some x / None</code> ) |

---

## Part 1 — on paper

### Exercise 1 (the message)

Write OCaml types for the following, choosing *sum* or *product* as the facts demand. An **address** is a user name and a domain, both text. A **channel** is one of three: email, to an address; SMS, to a phone number; push, to a device token (both text). A **message** has a channel, a body, and a subject — which an SMS does not have. Then draw one concrete message sent by email as a tree: constructors at the nodes, text at the leaves. (If you write `type phone = string`, note that this synonym invents no values: every string is a phone. A sum does invent values — *Email*, *Sms*, *Push* did not exist before.)

**Answer.**

```
type address = { user : string; domain : string }
type channel = Email of address | Sms of string | Push of string
type message = { to_ : channel; subject : string option; body : string }
```

Address and message are products (every field is always present); channel is a sum (exactly one of three kinds, each carrying its own kind of destination — the payloads differ, which is why a sum and not a record with a “kind” field). The subject is `string option`, not an empty string: the type says it can be absent, and every function that reads it has to face that case. The tree has message at the root; below it `Email` with an `address` node holding two text leaves, `Some "..."` or `None`, and the body text.

### Exercise 2 (why “algebraic”)

Let `type priority = Low | Normal | High`. How many distinct values are there of each type: `bool * bool`; `priority`; `priority * bool`; `priority option`; `bool option option`; `(priority * bool) option`? State the two rules that give the answers.

**Answer.** 4; 3; 6; 4; 3; 7. A *sum* has as many values as its summands added: `option` adds one (`None`) to what it wraps, so `priority option` has 3+1 and `bool option option` has (2+1)+1. A *product* multiplies: `priority * bool` has 3 × 2, and wrapping it in `option` gives 6+1. That

is the algebra in the name: a bare constructor counts 1, `|` is `+`, `*` and record fields are `×`. (A recursive type, or one containing `string`, has infinitely many values — the rules still say how each is built.)

### Exercise 3 (missing and unreachable)

*Two functions on `channel`, written in a hurry:*

```
let destination c = match c with
| Email a -> a.user ^ "@" ^ a.domain
| Sms number -> number

let is_push c = match c with
| Email _ | Sms _ | Push _ -> false
| Push _ -> true
```

*For each: does it compile? What does the compiler print, and what is the message called? What does `destination (Push t)` do at run time, and what does `is_push (Push t)` return? Fix both.*

**Answer.** Both compile — with a warning each, and the two warnings are the two dual checks of the lecture. The first is *Warning 8: this pattern-matching is not exhaustive. Here is an example of a case that is not matched: `Push _`*; at run time `destination (Push t)` raises `Match_failure`. The second is *Warning 11: this match case is unused*, on the arm `Push _ -> true`: the or-pattern above already takes every `Push`, so `is_push (Push t)` returns `false` — no crash, only a wrong answer, and only the warning reveals it. Fixes: add `| Push token -> token`; and take `Push _` out of the or-pattern (or move its arm above it). Arms are tried top to bottom, and the first that fits wins.

### Exercise 4 (the grid)

*Three channels, `Email`, `Sms`, `Push`, and two things to do with each: `send` and `format`. Six cells of code. Draw the table, channels as rows and operations as columns. In OCaml, which cells sit together in one function? In Java, with one class per channel, which sit together in one class? Now the boss asks for two changes, separately: add WhatsApp as a channel, and add `schedule` as an operation. For each change, in each language: how many new cells, how many existing functions or classes are touched — and what does the compiler say if a cell is forgotten?*

**Answer.** In OCaml a *column* is one function: `send` is one `match` holding the `Email`, `Sms` and `Push` cells; `format` the other three. In Java a *row* is one class: `Email` holds its own `send` and `format`. Adding WhatsApp adds a row: in OCaml two new cells, one in each existing function, both touched; in Java one new class, nothing else touched. Adding `schedule` adds a column: in OCaml one new function, nothing else touched; in Java one abstract method and a body in each of the three classes, all touched. Each way makes one direction free and the other expensive — that is the expression problem, and neither side is a mistake.

The point that is easily missed: in *both* languages the expensive direction is *checked*. Forget WhatsApp in OCaml's `format` and the compiler prints Warning 8 with the missing case named, once per function; forget `schedule` in Java's `Push` and the class does not compile until the method exists. A closed list of shapes lets the compiler enumerate the cells. A `switch` on a string, or an `if` chain on a kind field, has the same layout and none of the checking. Session 10 is about the ways to make both directions cheap at once.

## Part 2 — at the keyboard: warmup.ml

The file `warmup/warmup.ml` holds five small types with holes marked `TODO`, and a check for each; the exercises are stated here, the file only says where to type. It needs no `dune`:

```
ocaml warmup/warmup.ml          # pass / FAIL / todo, one line per check
```

Each rung is a few lines, and each is one match on one type.

### Exercise 5 (a variant)

*type weekday = Mon | Tue | Wed | Thu | Fri | Sat | Sun. Write next : weekday -> weekday (Sunday wraps to Monday) with one arm per day and no wildcard, and is\_weekend : weekday -> bool with an or-pattern. Why prefer seven arms over | \_ ->?*

**Answer.**

```
let next d = match d with
| Mon -> Tue | Tue -> Wed | Wed -> Thu | Thu -> Fri
| Fri -> Sat | Sat -> Sun | Sun -> Mon
let is_weekend d = match d with
| Sat | Sun -> true
| Mon | Tue | Wed | Thu | Fri -> false
```

With seven arms, a day added to the type would make the compiler point here; a wildcard would accept it silently.

### Exercise 6 (a record)

*type point = { x : float; y : float }. Write dist : point -> point -> float and move\_up : point -> float -> point, the latter with { p with y = ...}. After move\_up p 1., what is p.y?*

**Answer.**

```
let dist p q = let dx = p.x -. q.x and dy = p.y -. q.y in sqrt (dx *. dx +. dy *. dy)
let move_up p dy = { p with y = p.y +. dy }
```

p.y is unchanged: the functional update builds a new record; no field is mutable, so nothing could change the old one. Part C1 of the lab does exactly this to a pitch's octave.

### Exercise 7 (payloads, and a constructor added)

*type shape = Circle of float | Rect of float \* float. Write area and perimeter. When both checks pass, add | Square of float to the type and run the file again without touching the functions. Copy down what the compiler prints. Then fix the functions, guided by what it printed.*

**Answer.**

```
let area s = match s with
| Circle r -> pi *. r *. r | Rect (w, h) -> w *. h | Square a -> a *. a
let perimeter s = match s with
| Circle r -> 2. *. pi *. r | Rect (w, h) -> 2. *. (w +. h) | Square a -> 4. *. a
```

Before the Square arms exist, the compiler prints, for area and again for perimeter: *Warning 8: this pattern-matching is not exhaustive. Here is an example of a case that is not matched: Square ..* Two sites, found without searching — exercise 4's grid in code. The lab's Part B2 does the same on six functions.

### Exercise 8 (options)

*Write safe\_div : int -> int -> int option (None on division by zero) and max\_of : int list -> int option (None on the empty list). In max\_of, the recursive call returns an option: match on it.*

**Answer.**

```

let safe_div a b = if b = 0 then None else Some (a / b)
let rec max_of xs = match xs with
| [] -> None
| x :: rest -> (match max_of rest with
| None -> Some x
| Some m -> Some (if x > m then x else m))

```

'a option is “an 'a, or nothing”: two constructors, and the caller must match on them; there is no null value that a caller could forget to test.

### Exercise 9 (a tree)

*type expr = Num of int / Add of expr \* expr / Mul of expr \* expr. Write eval : expr -> int and to\_string : expr -> string so that Add (Num 1, Mul (Num 2, Num 3)) prints 1 + 2 \* 3 and Mul (Add (Num 1, Num 2), Num 3) prints (1 + 2) \* 3. When does a sum need parentheses? This tree — leaves and binary nodes — is the shape of the lab's melody, and the parentheses question returns in Part C2.*

**Answer.**

```

let rec eval e = match e with
| Num n -> n | Add (a, b) -> eval a + eval b | Mul (a, b) -> eval a * eval b

let rec to_string e =
  let factor f = match f with Add _ -> "(" ^ to_string f ^ ")" | _ -> to_string f in
  match e with
  | Num n -> string_of_int n
  | Add (a, b) -> to_string a ^ " + " ^ to_string b
  | Mul (a, b) -> factor a ^ " * " ^ factor b

```

A sum needs parentheses only inside a product, so the Mul arm looks at the *shape* of its children — a nested pattern — and wraps an Add. In the music language, Par is the node that must bring its own brackets.